LAB 03 — Word Representations and Embeddings

Bài toán: xây dựng biểu diễn từ dense từ thống kê đồng xuất hiện (co-occurrence) và Word2Vec; đánh giá bằng similarity, analogy và semantic search. Pipeline: `context → co-occurrence matrix → PPMI → (SVD) → cosine → Word2Vec (CBOW / Skip-gram) → WordSim-353, analogy → semantic search`.

Cấu trúc thư mục

File	Nội dung
`report.pdf`	Bản tổng hợp: README, calculations, prediction, word_embedding.ipynb, error_analysis, reflection
`calculations.md`	Bài 1 (co-occurrence), Bài 2–3 (cosine), Bài 4 (sparse vs dense), CBOW/Skip-gram training pairs, analogy
`prediction.md`	4 dự đoán trước khi chạy model
`cooccurrence.py`	`build_vocabulary`, `build_cooccurrence_matrix`, `ppmi`, `cosine_similarity`, `most_similar` (tự cài đặt) + 5 unit tests
`word_embedding.ipynb`	Co-occurrence (window 1/2/5, raw vs PPMI), Word2Vec CBOW vs Skip-gram, nearest neighbors, siêu tham số (window, dimension), IMDb downstream, WordSim-353, analogy, semantic search, polysemy *bank* (đã chạy)
`results.csv`	Toàn bộ metric của các thí nghiệm
`error_analysis.md`	3 similarity đúng, 3 sai/bất ngờ, phân loại nguyên nhân
`reflection.md`	Đối chiếu prediction, bảng representation, bank cần contextual embedding, bài học

Thiết lập

- **Corpus:** text8 (17 005 207 token Wikipedia, lowercase, không dấu câu, 253 854 từ khác nhau).
- **Co-occurrence (tự cài):** 2 000 000 token đầu, vocabulary 10 000 từ phổ biến nhất; window đối xứng, không vượt qua ranh giới câu.
- **Word2Vec:** `gensim` 100 chiều, min_count 5 (71 290 từ), 5 epoch, negative sampling 5, subsampling 1e-3, seed 42. CBOW (`sg=0`) và Skip-gram (`sg=1`).
- **Đánh giá:** WordSim-353 (Spearman ρ) và Google analogy (19 544 câu, `questions-words.txt`).
- **Semantic search:** corpus 30K arXiv của LAB 01, Word2Vec-arXiv train trên 113K abstracts (không dùng nhãn), so sánh với TF-IDF bằng `qrels.json` của LAB 01.
- **Downstream (Exp 4):** 5 000 review IMDb train / 5 000 test, document = trung bình word vector → Logistic Regression.

Kết quả chính

Co-occurrence (Exp 1): window 1 cho *doctor* → *physician* là gần nhất (sau PPMI) trong khi window 5 cho *doctor* → *patient, medical*. Raw count cho cosine cao chung chung (*king–queen* 0.93–0.97); PPMI làm giảm nhiễu từ phổ biến.

Architecture (Exp 2, window 5, dim 100):

Model	WordSim-353 ρ	Analogy acc	Train time
CBOW	0.622	0.312	26 s

Model	WordSim-353 ρ	Analogy acc	Train time
Skip-gram	**0.689**	**0.386**	52 s

Window (Exp 3): window 2 / 5 / 10 → WordSim 0.637 / 0.689 / 0.691; analogy 0.302 / 0.386 / 0.410. Window lớn hơn tăng analogy nhưng làm *doctor* gần nhân vật truyền hình hơn.

Dimension (Exp 4): 50 / 100 / 300 → WordSim 0.662 / 0.689 / 0.676; IMDb accuracy (avg embedding + LR) 0.723 / 0.754 / 0.785. Thời gian train 55 s / 52 s / 207 s.

Semantic search (LAB 01 corpus, 8 query):

Phương pháp	P@5	MRR
TF-IDF	0.525	0.642
Query expansion (Word2Vec)	0.250	0.321
Avg embedding (Word2Vec-arXiv)	0.300	0.664
Hybrid (0.5 TF-IDF + 0.5 avg emb.)	0.475	0.664

Tóm lại: embedding cải thiện MRR nhưng làm giảm P@5 so với TF-IDF; Hybrid giữ được gần độ chính xác TF-IDF với MRR tốt hơn. Query expansion không có lợi trong thiết lập này.

Cách chạy lại

```
pip install gensim scikit-learn datasets pandas matplotlib scipy nbconvert ipykernel
cd lab3
python cooccurrence.py                                                    # unit tests
jupyter nbconvert --to notebook --execute --inplace word_embedding.ipynb  # ~10 phút, cần text8
(gensim-data) và IMDb
```

AI assistance statement

- **Tool:** Claude Code
- **Purpose:** viết code (`cooccurrence.py`, `word_embedding.ipynb`), tạo biểu đồ và `results.csv`; trình bày các file markdown và ghép `report.pdf`.
- **Generated:** code, biểu đồ, `results.csv`, `report.pdf`, bản nháp các file markdown.
- **Student:** bài tính tay, prediction, rà soát giải thích các similarity trong error analysis và reflection.
- **Verified:** 5 unit tests pass; bài tính tay khớp với `cooccurrence.py` (cos(cat, dog) = 1, cos(doctor, physician) = 0.987); số liệu trong các file `.md` lấy từ output notebook.

Thứ ngày . .

Ex3:

Bài 1:

Liệt kê các context (±1) của các target

Word	S1	S2	S3	S4
cat	the, eats	—	the, likes	—
dog	—	the, eats	—	the, likes
eats	cat, fish	dog, fish	—	—
likes	—	—	cat, milk	dog, meat

Đếm theo Vocabulary, ko lấy the làm cột

Word	cats	dogs	eats	likes	fish	milk	meat
cat	0	0	1	1	0	0	0
dog	0	0	1	1	0	0	0
eats	1	1	0	0	2	0	0
like	1	1	0	0	0	1	1

cat = [0,0,1,1,0,0,0]

dog = [0,0,1,1,0,0,0]

likes = [1,1,0,0,2,0,0]

eats = [1,1,0,0,0,1,1]

vec(cat) = vec(dog) ⇒ cos(cat, dog) = 1

$\cos(\text{eats, likes}) = (1.1+1.1)/(\sqrt{1+1+4}).(\sqrt{1+1+1+1})$

$= 2/(\sqrt{6}.2) = 2/4,899 \approx 0,408$

cùng chủ ngữ cat, dog, khác tân ngữ

Bài 2:

$\cos(x,y) = \frac{x.y}{|x|.|y|} = \frac{1.2+2.4+1.2}{\sqrt{1+4+1}.\sqrt{4+16+4}} = \frac{12}{12} = 1$

cos = 1 vì ~~x~~ y = 2x, cosine chỉ đo hướng nên = 1

cos = 1 là 2 từ có cùng tỉ lệ context

Scanned with CamScanner

Thứ ngày . .

Bài 3:

doctor = [0.8, 0.1, 0.7]

physician = [0.7, 0.2, 0.8]

banana = [0.2, 0.9, 0.1]

dự đoán physician gần hơn (cùng lớn ở chiều 1, 3)

$\cos(doc, phy) = \frac{1.14}{\sqrt{1.14}\cdot\sqrt{1.7}} \approx 0,9871$

$\cos(doc, banana) = \frac{-0,14}{\sqrt{0,86}\cdot\sqrt{1.41}} \approx -0,1414$

⇒ dự đoán đúng

Bài 4: giay

1) Sparse: word-context representation 10000 chiều, chỉ 30 phần tử ≠ 0

2) Dense: embedding 300 chiều, hầu hết các thành phần ≠ 0

3) Vì sao dense thuần lợi hơn cho sematic similarity.

Vì: trong vector spare mỗi chiều là 1 context world cụ thể; 2 từ đồng nghĩa nhưng thấy context khác chữ sẽ không có chiều chung ⇒ cosine ≈ 0. Dense embedding nén các context có liên quan vào cùng các chiều ẩn, nên similarity đc chia sẻ qua những từ chưa xuất hiện chung

3) ko chắc tốt hơn: do dense dễ làm từ trái nghĩa gần nhau (cùng ngữ cảnh) và cần nhiều dữ liệu huấn luyện

- sparse hiệu quả hơn với bài toán cần tìm chính xác từ khoá

Scanned with CamScanner

Thứ ngày . .

Bài 5: mục 1 c.

the cat eats fish window=1.

CBOW (context -> target)

	Input (context)	Target
1	[cat]	the
2	[the, eats]	cat
3	[cat, fish]	eats
4	[eats]	fish.

Skip-gram (target -> context word)

	Input	output (context)
1	the	cat
2	cat	the
3	cat	eats.
4	eats	cat
5	eats	fish
6	fish	eats

-> CBOW: dùng từ ngữ cảnh đoán từ trung tâm (tạo ít dữ liệu huấn luyện, train nhanh hơn)

-> Skip-gram: dùng từ trung tâm dự đoán từ ngữ cảnh -> tạo nhiều ví dụ, học từ hiếm tốt, nhưng train chậm hơn.

Bài 6:

King - man + women = [8, 4, 7]

ý nghĩa: vector mới giữ địa vị, đổi giới tính

LAB 03 — Word Representations and Embeddings — Trang 5/19
Scanned with CamScanner

Prediction (ghi trước khi chạy model)

Corpus dự kiến: text8 (17 triệu token Wikipedia) cho Word2Vec; 2 triệu token đầu cho co-occurrence.

Prediction 1 — Những từ nào gần nhau nhất? (doctor, physician, hospital, banana, car)

Prediction: doctor–physician gần nhau nhất, sau đó là doctor–hospital và physician–hospital. banana và car xa ba từ y tế; banana–car cũng xa nhau.

Reason: doctor và physician thay thế được cho nhau trong cùng câu (*the ___ treated the patient*) nên có context gần như giống nhau. hospital xuất hiện *cùng* doctor (quan hệ chủ đề) chứ không thay thế được doctor, nên gần nhưng kém hơn. banana và car thuộc chủ đề khác hẳn.

Confidence: Cao

Prediction 2 — Context window tăng từ 2 lên 5, similarity có thay đổi không?

Prediction: Có. Window nhỏ ưu tiên từ **cùng chức năng** (thay thế được: doctor–physician, cat–dog); window lớn ưu tiên từ **cùng chủ đề** (doctor–hospital, doctor–patient). Vậy khi tăng window, doctor–hospital tăng còn doctor–physician và cat–dog có thể giảm.

Reason: Với window ±2, context chủ yếu là từ ngữ pháp sát bên (mạo từ, động từ). Với window ±5, context gồm cả các từ nội dung cùng đoạn văn, nên hai từ thường đi chung một chủ đề cũng có context giống nhau.

Confidence: Vừa

Prediction 3 — Dimension 50 → 100 → 300, chất lượng có chắc chắn tăng không?

Prediction: Không chắc chắn. 50 → 100 có thể tăng; 100 → 300 tăng ít hoặc không tăng trên similarity/analogy với corpus 17M token, trong khi thời gian train và kích thước model tăng khoảng 3 lần.

Reason: Nhiều chiều hơn cho nhiều capacity hơn, nhưng cần nhiều dữ liệu hơn để học hết các tham số. Corpus không đủ lớn thì các chiều thêm vào chủ yếu học nhiễu. Bài toán downstream có thể vẫn hưởng lợi vì classifier có nhiều đặc trưng hơn.

Confidence: Vừa

Prediction 4 — doctor và physician có chắc chắn gần nhau nếu corpus chỉ có 100 câu?

Prediction: Không. Với 100 câu, mỗi từ chỉ xuất hiện vài lần; vector gần như phụ thuộc vào khởi tạo ngẫu nhiên và vài context tình cờ, nên cosine có thể cao hay thấp tùy lần chạy và nearest neighbors không có ý nghĩa.

Reason: Distributional hypothesis là một phát biểu thống kê: cần nhiều quan sát context thì mới ước lượng được phân bố context đáng tin.

Confidence: Cao

LAB 03 — Word Representations and Embeddings (experiments)

Corpus chính: `text8` — 17 005 207 token đầu của Wikipedia tiếng Anh (đã lowercase, bỏ dấu câu), qua `gensim-data`.

- Experiment 1 (co-occurrence, tự cài đặt): 2 000 000 token đầu, vocabulary 10 000 từ phổ biến nhất.
- Word2Vec (Experiment 2–4): toàn bộ text8.
- Application (semantic search): corpus 30K arXiv ML abstracts của LAB 01 + `qrels.json` của LAB 01; Word2Vec được train thêm trên 113K arXiv abstracts (cùng domain).

Đánh giá định lượng: **WordSim-353** (Spearman ρ giữa cosine và điểm của người) và **Google analogy** (`questions-words.txt`, 19 544 câu hỏi), cả hai có sẵn trong `gensim.test.utils`.

```
import os, warnings
os.environ["HF_HUB_DISABLE_PROGRESS_BARS"] = "1"; os.environ["HF_HUB_VERBOSITY"] = "error"
warnings.filterwarnings("ignore")
import gzip, json, re, time, logging
import numpy as np, pandas as pd, scipy.sparse as sp, matplotlib.pyplot as plt
from collections import Counter
from gensim.models import Word2Vec
from gensim.test.utils import datapath
from sklearn.decomposition import TruncatedSVD
import cooccurrence as co

logging.getLogger("gensim").setLevel(logging.ERROR)
SEED = 42
words = gzip.open(os.path.expanduser("~/gensim-data/text8/text8.gz"), "rt").read().split()
text8 = [words[i:i + 10000] for i in range(0, len(words), 10000)]      # "câu" 10 000 token như gensim
freq = Counter(words)
print(f"text8: {len(words):,} token, {len(freq):,} từ khác nhau, {len(text8)} chunks")
WORDS = ["doctor", "physician", "hospital", "patient", "disease", "computer", "football", "banana", "car",
"automobile",
         "king", "queen", "cat", "dog", "bank"]
print({w: freq[w] for w in WORDS})

STOP = set("the of and in a to is as was by for on that with from at it an are his which be or this also has had
were not but its their have one two three four five six seven eight nine zero he she they who".split())
pos_index = {}
def context_evidence(word, window=5, top=10):
    """Các context word (bỏ stopwords) xuất hiện nhiều nhất quanh word trong text8 — bằng chứng từ corpus."""
    if word not in pos_index:
        pos_index[word] = np.array([i for i, w in enumerate(words) if w == word])
    c = Counter()
    for i in pos_index[word]:
        c.update(w for w in words[max(0, i - window):i] + words[i + 1:i + 1 + window] if w not in STOP)
    return c.most_common(top)
```

Kiểm chứng bài tính tay (`calculations.md`)

```
co.run_tests()
BAI1 = [s.split() for s in ["the cat eats fish", "the dog eats fish", "the cat likes milk", "the dog likes meat"]]
V7 = ["cat", "dog", "eats", "likes", "fish", "milk", "meat"]
X7 = co.build_cooccurrence_matrix(BAI1, V7, window=1).toarray().astype(int)
display(pd.DataFrame(X7[:4], index=V7[:4], columns=V7))
print("cos(cat, dog) =", co.cosine_similarity(X7[0], X7[1]), "| cos(eats, likes) =",
round(co.cosine_similarity(X7[2], X7[3]), 4))
print("Bài 2: cos([1,2,1],[2,4,2]) =", co.cosine_similarity([1, 2, 1], [2, 4, 2]))
doctor, physician, banana = [0.8, 0.1, 0.7], [0.7, 0.2, 0.8], [-0.2, 0.9, -0.1]
print(f"Bài 3: cos(doctor, physician) = {co.cosine_similarity(doctor, physician):.4f}, cos(doctor, banana) =
```

```
{co.cosine_similarity(doctor, banana):.4f}")
print("Bài analogy: king - man + woman =", (np.array([8, 2, 7]) - np.array([5, 1, 5]) + np.array([5, 3,
5])).tolist())

def cbow_pairs(sent, w=1):
    return [(sent[max(0, i - w):i] + sent[i + 1:i + 1 + w], t) for i, t in enumerate(sent)]
def skipgram_pairs(sent, w=1):
    return [(t, c) for ctx, t in cbow_pairs(sent, w) for c in ctx]
print("CBOW     :", cbow_pairs("the cat eats fish".split()))
print("Skip-gram:", skipgram_pairs("the cat eats fish".split()))
```

Experiment 1 — Word-context representation (co-occurrence, tự cài đặt)

2M token đầu của text8, vocabulary = 10 000 từ phổ biến nhất (rows = target, columns = context). So sánh raw count và **PPMI** (giảm trọng số context phổ biến như *the, of*).

```
small = [words[:2_000_000]]
vocab10k = co.build_vocabulary(small, max_size=10000)
widx = {w: i for i, w in enumerate(vocab10k)}
PAIRS = [("doctor", "physician"), ("doctor", "hospital"), ("doctor", "patient"), ("doctor", "music"), ("cat",
"dog"),
         ("king", "queen"), ("car", "automobile"), ("computer", "banana"), ("computer", "river")]
print("Bỏ các cặp có từ ngoài vocabulary 10K:", [p for p in PAIRS if not all(w in widx for w in p)])
PAIRS = [p for p in PAIRS if all(w in widx for w in p)]
exp1, matrices = [], {}
for w in (1, 2, 5):
    t0 = time.time()
    X = co.build_cooccurrence_matrix(small, vocab10k, window=w); P = co.ppmi(X)
    matrices[w] = (X, P)
    row = {"window": w, "vocabulary": len(vocab10k), "matrix size": f"{X.shape[0]:,} × {X.shape[1]:,}", "non-zero":
X.nnz,
           "density": X.nnz / (X.shape[0] * X.shape[1]), "time (s)": round(time.time() - t0, 2)}
    for a, b in PAIRS:
        row[f"{a}–{b} (count)"] = co.cosine_similarity(X[widx[a]], X[widx[b]])
        row[f"{a}–{b} (PPMI)"] = co.cosine_similarity(P[widx[a]], P[widx[b]])
    exp1.append(row)
exp1 = pd.DataFrame(exp1).set_index("window")
display(exp1.iloc[:, :5])
display(exp1.iloc[:, 5:].T.round(3))
```

```
rows = []
for w in (1, 2, 5):
    X, P = matrices[w]
    rows.append({"window": w, "raw count": ", ".join(f"{a} {s:.2f}" for a, s in co.most_similar("doctor", X,
vocab10k, 5)),
                 "PPMI": ", ".join(f"{a} {s:.2f}" for a, s in co.most_similar("doctor", P, vocab10k, 5))})
print("most_similar('doctor') — co-occurrence tự cài đặt")
display(pd.DataFrame(rows).set_index("window"))
# Từ co-occurrence sparse → dense: TruncatedSVD của PPMI (window 5) xuống 100 chiều
svd = TruncatedSVD(n_components=100, random_state=SEED)
D = svd.fit_transform(matrices[5][1])
print(f"PPMI window 5: {matrices[5][1].shape[1]:,} chiều sparse (density {matrices[5][1].nnz / 1e8:.2%}) → SVD 100
chiều dense")
for word in ("doctor", "king", "computer"):
    print(f"  SVD-100 most_similar({word!r}):", [(a, round(s, 2)) for a, s in co.most_similar(word, D, vocab10k,
5)])
```

Experiment 2 — Word2Vec trên text8

Tham số	Giá trị
Corpus	text8, 17 005 207 token
Vocabulary	từ có count ≥ `min_count` = 5 → 71 290 từ

Tham số	Giá trị
Embedding dimension (`vector_size`)	100
Context window	5
Epochs	5
Training objective	CBOW (`sg=0`) và Skip-gram (`sg=1`), cả hai dùng **negative sampling** (5 negative samples), subsampling `sample=1e-3`
Learning rate	0.025 → 0.0001 (tuyến tính)
Khác	`workers=12` , `seed=42` (đa luồng nên kết quả có sai khác nhỏ giữa các lần chạy)

```
ws353 = pd.read_csv(datapath("wordsim353.tsv"), sep="\t", comment="#", header=None, names=["w1", "w2", "score"])
def evaluate(m):
    sim = m.wv.evaluate_word_pairs(datapath("wordsim353.tsv"))
    ana_score, sections = m.wv.evaluate_word_analogies(datapath("questions-words.txt"), restrict_vocab=30000)
    sem = [s for s in sections if not s["section"].startswith("gram") and s["section"] != "Total accuracy"]
    syn = [s for s in sections if s["section"].startswith("gram")]
    acc = lambda ss: sum(len(s["correct"]) for s in ss) / max(1, sum(len(s["correct"]) + len(s["incorrect"]) for s
in ss))
    return {"WordSim353 Spearman": sim[1][0], "WordSim353 OOV %": sim[2], "analogy acc": ana_score,
            "analogy semantic": acc(sem), "analogy syntactic": acc(syn)}

def train(**kw):
    cfg = dict(vector_size=100, window=5, min_count=5, sg=0, epochs=5, negative=5, sample=1e-3, workers=12,
seed=SEED)
    cfg.update(kw)
    t0 = time.time(); m = Word2Vec(text8, **cfg); dt = time.time() - t0
    size_mb = (m.wv.vectors.nbytes + m.syn1neg.nbytes) / 1e6
    return m, {"train time (s)": dt, "vocab": len(m.wv), "params": m.wv.vectors.size + m.syn1neg.size, "model size
(MB)": size_mb}

models, info = {}, {}
for name, kw in [("CBOW", {"sg": 0}), ("Skip-gram", {"sg": 1})]:
    models[name], info[name] = train(**kw)
    info[name].update(evaluate(models[name]))
arch = pd.DataFrame(info).T
display(arch.round(4))
```

```
MAIN = "Skip-gram" if arch.loc["Skip-gram", "WordSim353 Spearman"] + arch.loc["Skip-gram", "analogy acc"] >= \
                      arch.loc["CBOW", "WordSim353 Spearman"] + arch.loc["CBOW", "analogy acc"] else "CBOW"
wv = models[MAIN].wv
print(f"Model chính (WordSim + analogy tốt hơn): {MAIN}")
INSPECT = ["doctor", "hospital", "patient", "disease", "computer", "football", "banana"]
display(pd.DataFrame({w: [f"{a} ({s:.2f})" for a, s in wv.most_similar(w, topn=5)] for w in INSPECT}, index=[f"top-
{i}" for i in range(1, 6)]).T)
print("CBOW most_similar('doctor'):", [(a, round(s, 2)) for a, s in models["CBOW"].wv.most_similar("doctor",
topn=5)])
```

Bằng chứng từ corpus: vì sao các từ này gần *doctor*?

Context word (±5, bỏ stopwords) xuất hiện nhiều nhất quanh từng từ trong text8.

```
for w in ["doctor", "physician"] + [a for a, _ in wv.most_similar("doctor", topn=2)]:
    print(f"{w:12s} (count {freq[w]:5d}):", context_evidence(w, top=12))
bigr = Counter(zip(words, words[1:]))
print("\nCác cụm có 'doctor':", sorted(((c, f"{a} {b}") for (a, b), c in bigr.items() if b == "doctor" or a ==
"doctor"), reverse=True)[:12])
```

Prediction 1 & 4 — similarity giữa 5 từ, và corpus chỉ có 100 câu

Corpus nhỏ: 100 "câu" (cửa sổ 21 token) cắt quanh 50 lần xuất hiện của *doctor* và 50 lần của *physician* trong text8 (≈ 2 100 token, thiên vị có lợi cho hai từ này). Train Skip-gram (100 chiều, window 5,

min_count 1, 50 epochs) với 5 seed khác nhau.

```
five = ["doctor", "physician", "hospital", "banana", "car"]
display(pd.DataFrame([[wv.similarity(a, b) for b in five] for a in five], index=five, columns=five).round(3))
rng = np.random.RandomState(SEED)
pos = {w: rng.choice(np.where(np.array(words[:5_000_000]) == w)[0], 50, replace=False) for w in ("doctor",
"physician")}
tiny = [words[i - 10:i + 11] for w in pos for i in pos[w]]
tiny_rows = []
for seed in range(5):
    m = Word2Vec(tiny, vector_size=100, window=5, min_count=1, sg=1, epochs=50, workers=1, seed=seed)
    tiny_rows.append({"seed": seed, "vocab": len(m.wv), "cos(doctor, physician)": m.wv.similarity("doctor",
"physician"),
                      "top-3 of doctor": [a for a, _ in m.wv.most_similar("doctor", topn=3)]})
tiny_df = pd.DataFrame(tiny_rows)
display(tiny_df.round(3))
print(f"100 câu: cos(doctor, physician) = {tiny_df.iloc[:, 2].mean():.3f} ± {tiny_df.iloc[:, 2].std():.3f} (5
seeds) | text8 đầy đủ: {wv.similarity('doctor', 'physician'):.3f}")
```

Experiment 3 — Context window (2 / 5 / 10)

```
EVAL_PAIRS = [("doctor", "physician"), ("doctor", "hospital"), ("doctor", "nurse"), ("cat", "dog"), ("car",
"automobile"),
              ("king", "queen"), ("computer", "banana"), ("good", "bad")]
win_models, win_info = {}, {}
for w in (2, 5, 10):
    if w == 5:
        win_models[w], win_info[w] = models[MAIN], dict(info[MAIN])
    else:
        win_models[w], win_info[w] = train(sg=int(MAIN == "Skip-gram"), window=w)
        win_info[w].update(evaluate(win_models[w]))
    for a, b in EVAL_PAIRS:
        win_info[w][f"{a}-{b}"] = win_models[w].wv.similarity(a, b)
win_table = pd.DataFrame(win_info).T
win_table.index.name = "window"
display(win_table[["train time (s)", "WordSim353 Spearman", "analogy semantic", "analogy syntactic", "analogy
acc"]].round(4))
display(win_table[[f"{a}-{b}" for a, b in EVAL_PAIRS]].T.astype(float).round(3))
for w in (2, 10):
    print(f"window {w:2d} most_similar('doctor'):", [(a, round(s, 2)) for a, s in
win_models[w].wv.most_similar("doctor", topn=6)])
```

Experiment 4 — Embedding dimension (50 / 100 / 300)

Downstream task đơn giản: sentiment IMDb (5 000 review train / 5 000 review test lấy ngẫu nhiên), document = trung bình word vectors → Logistic Regression.

```
from datasets import load_dataset
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
imdb = load_dataset("stanfordnlp/imdb")
tr = imdb["train"].shuffle(seed=SEED).select(range(5000)); te = imdb["test"].shuffle(seed=SEED).select(range(5000))
tok = lambda t: re.findall(r"[a-z]+", t.lower().replace("<br />", " "))
tr_tok, te_tok = [tok(t) for t in tr["text"]], [tok(t) for t in te["text"]]

def downstream(kv):
    emb = lambda docs: np.vstack([np.mean([kv[w] for w in d if w in kv.key_to_index] or [np.zeros(kv.vector_size)],
axis=0) for d in docs])
    sc = StandardScaler().fit(emb(tr_tok))
    clf = LogisticRegression(max_iter=2000, C=1.0).fit(sc.transform(emb(tr_tok)), tr["label"])
    return clf.score(sc.transform(emb(te_tok)), te["label"])

dim_models, dim_info = {}, {}
for d in (50, 100, 300):
    if d == 100:
        dim_models[d], dim_info[d] = models[MAIN], dict(info[MAIN])
```

```
    else:
        dim_models[d], dim_info[d] = train(sg=int(MAIN == "Skip-gram"), vector_size=d)
        dim_info[d].update(evaluate(dim_models[d]))
    dim_info[d]["IMDb accuracy (avg emb + LR)"] = downstream(dim_models[d].wv)
    for a, b in EVAL_PAIRS[:6]:
        dim_info[d][f"{a}-{b}"] = dim_models[d].wv.similarity(a, b)
dim_table = pd.DataFrame(dim_info).T
dim_table.index.name = "dimension"
display(dim_table[["train time (s)", "params", "model size (MB)", "WordSim353 Spearman", "analogy acc", "IMDb
accuracy (avg emb + LR)"]].round(4))
display(dim_table[[f"{a}-{b}" for a, b in EVAL_PAIRS[:6]]].T.astype(float).round(3))
fig, ax = plt.subplots(2, 3, figsize=(12, 6))
for r, (tab, xl) in enumerate([(win_table, "window (dimension 100)"), (dim_table, "dimension (window 5)")]):
    for c, col in enumerate(["train time (s)", "WordSim353 Spearman", "analogy acc"]):
        ax[r, c].plot(tab.index, tab[col].astype(float), marker="o", color=["#4c72b0", "#dd8452"][r])
        ax[r, c].set_title(col); ax[r, c].set_xlabel(xl); ax[r, c].set_xticks(list(tab.index))
plt.tight_layout(); plt.savefig("hyperparameters.png", dpi=150); plt.show()
```

Evaluation — Word similarity và analogy

```
sim_rows = [{"pair": f"{a} – {b}", "cosine": wv.similarity(a, b)} for a, b in
            [("doctor", "physician"), ("car", "automobile"), ("king", "queen"), ("cat", "dog"), ("computer",
"banana")]]
sim_eval = pd.DataFrame(sim_rows).sort_values("cosine", ascending=False).reset_index(drop=True)
sim_eval.index += 1
display(sim_eval.round(3))
ws = ws353.copy()
ws["cosine"] = [wv.similarity(a, b) if a in wv and b in wv else np.nan for a, b in zip(ws.w1, ws.w2)]
ws = ws.dropna(); ws["human rank"] = ws.score.rank(ascending=False); ws["model rank"] =
ws.cosine.rank(ascending=False)
ws["rank gap"] = ws["model rank"] - ws["human rank"]
print(f"WordSim-353: Spearman ρ = {ws[['score', 'cosine']].corr('spearman').iloc[0, 1]:.3f} trên {len(ws)} cặp")
print("Cặp model đánh giá CAO hơn người nhiều nhất:"); display(ws.sort_values("rank gap").head(5).round(3))
print("Cặp model đánh giá THẤP hơn người nhiều nhất:"); display(ws.sort_values("rank gap",
ascending=False).head(5).round(3))
```

```
ANALOGIES = [("king", "man", "woman", "queen"), ("paris", "france", "italy", "rome"), ("walking", "walk", "swim",
"swimming"),
             ("bigger", "big", "small", "smaller"), ("brother", "man", "woman", "sister"), ("doctor", "man",
"woman", "nurse")]
ana_rows = []
for a, b, c, exp in ANALOGIES:
    top = wv.most_similar(positive=[a, c], negative=[b], topn=5)
    rank = next((i for i, (w, _) in enumerate(top, 1) if w == exp), None)
    ana_rows.append({"query": f"{a} – {b} + {c}", "expected": exp, "top-5": ", ".join(f"{w} {s:.2f}" for w, s in
top), "rank of expected": rank or ">5"})
display(pd.DataFrame(ana_rows))
display(arch[["analogy acc", "analogy semantic", "analogy syntactic"]].round(4))
```

Application — Semantic search (nối tiếp LAB 01)

Corpus và nhãn relevance của LAB 01 (30K arXiv abstracts, 8 queries). Word2Vec text8 là domain Wikipedia, nên train thêm **Word2Vec-arXiv** (Skip-gram, 100 chiều, window 5, min_count 5, 5 epochs) trên toàn bộ 113K abstracts (không dùng nhãn nào).

Phương pháp	Mô tả
TF-IDF	Pipeline B của LAB 01 (baseline lexical)
Query expansion	Thêm 3 từ gần nhất (Word2Vec-arXiv) cho mỗi từ của query, trọng số = cosine, rồi tìm bằng TF-IDF
Avg embedding	Document/query = trung bình word vectors có trọng số IDF; cosine
Hybrid	0.5 · cosine TF-IDF + 0.5 · cosine avg embedding

```
from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
raw = load_dataset("CShorten/ML-ArXiv-Papers", split="train").to_pandas()[["title", "abstract"]].dropna()
for c in ["title", "abstract"]:
    raw[c] = raw[c].str.replace(r"\s+", " ", regex=True).str.strip()
raw = raw.drop_duplicates("title")
lab1 = raw.sample(n=30000, random_state=SEED).reset_index(drop=True)          # đúng corpus của LAB 01
lab1["doc_id"] = [f"D{i:05d}" for i in range(len(lab1))]
docs = (lab1.title + ". " + lab1.abstract).tolist()
norm_tok = lambda s: [w for w in re.sub(r"[^a-z0-9]+", " ", s.lower()).split() if w not in ENGLISH_STOP_WORDS]
t0 = time.time()
arxiv_sents = [norm_tok(t) for t in (raw.title + ". " + raw.abstract)]
w2v_arxiv = Word2Vec(arxiv_sents, vector_size=100, window=5, min_count=5, sg=1, epochs=5, workers=12, seed=SEED)
print(f"Word2Vec-arXiv: {sum(map(len, arxiv_sents)):,} token, vocab {len(w2v_arxiv.wv):,}, {time.time() -
t0:.0f}s")
for w in ["transformer", "attack", "car", "heart", "treatment", "doctor"]:
    print(f"  {w:12s} text8: {[a for a, _ in wv.most_similar(w, topn=5)]}\n  {'':12s} arXiv: {[a for a, _ in
w2v_arxiv.wv.most_similar(w, topn=5)]}")
```

```
qrels = {k: v for k, v in json.load(open("../lab1/qrels.json", encoding="utf-8")).items() if not k.startswith("_")}
id2idx = {d: i for i, d in enumerate(lab1.doc_id)}
tfv = TfidfVectorizer(tokenizer=norm_tok, lowercase=False, token_pattern=None)
XT = tfv.fit_transform(docs)
kv = w2v_arxiv.wv
idf = dict(zip(tfv.get_feature_names_out(), tfv.idf_))

def embed(tokens):
    vs = [(kv[w], idf.get(w, 1.0)) for w in tokens if w in kv.key_to_index]
    if not vs:
        return np.zeros(kv.vector_size)
    v = np.average([x for x, _ in vs], axis=0, weights=[w for _, w in vs])
    return v / (np.linalg.norm(v) or 1)

E = np.vstack([embed(norm_tok(d)) for d in docs])

def expand(query, k=3):
    terms = Counter({w: 1.0 for w in norm_tok(query)})
    for w in norm_tok(query):
        if w in kv.key_to_index:
            for nb, s in kv.most_similar(w, topn=k):
                terms[nb] = max(terms[nb], s)
    return terms

def scores(query, method):
    if method == "TF-IDF":
        return (XT @ tfv.transform([query]).T).toarray().ravel()
    if method == "Query expansion":
        terms = expand(query)
        q = tfv.transform([" ".join(terms)]).toarray().ravel()
        for w, wt in terms.items():
            if w in tfv.vocabulary_:
                q[tfv.vocabulary_[w]] *= wt
        return XT @ (q / (np.linalg.norm(q) or 1))
    if method == "Avg embedding":
        return E @ embed(norm_tok(query))
    return 0.5 * scores(query, "TF-IDF") + 0.5 * scores(query, "Avg embedding")

METHODS = ["TF-IDF", "Query expansion", "Avg embedding", "Hybrid"]
eval_rows = []
for m in METHODS:
    for qid, q in qrels.items():
        rel = {id2idx[d] for d in q["relevant"]}
        order = np.argsort(-scores(q["query"], m), kind="stable")
        first = next(r + 1 for r, i in enumerate(order) if i in rel)
        eval_rows.append({"method": m, "query_id": qid, "query": q["query"], "P@5": np.mean([i in rel for i in
order[:5]]),
                          "R@10": np.mean([i in rel for i in order[:10]]) * 10 / len(rel), "RR": 1 / first, "first
relevant rank": first})
search_eval = pd.DataFrame(eval_rows)
display(search_eval.groupby("method", sort=False)[["P@5", "R@10", "RR"]].mean().rename(columns={"RR":
"MRR"}).round(3))
display(search_eval.pivot(index="query", columns="method", values="first relevant rank")[METHODS])
```

```
def show(query, method, k=5):
    s = scores(query, method); order = np.argsort(-s)[:k]
    return pd.DataFrame({"rank": range(1, k + 1), "doc": lab1.doc_id[order].values, "score": s[order].round(3),
"title": lab1.title[order].str[:80].values})

q = "medical treatment"
print(f"Query {q!r} — expansion:", {w: round(v, 2) for w, v in expand(q).items()})
for m in ["TF-IDF", "Avg embedding"]:
    print(f"\n{m}:"); display(show(q, m))
q = "heart attack prediction"
print(f"Query {q!r} — expansion:", {w: round(v, 2) for w, v in expand(q).items()})
for m in ["TF-IDF", "Hybrid"]:
    print(f"\n{m}:"); display(show(q, m))
print("cos(heart attack, myocardial infarction) — Word2Vec-arXiv, trung bình vector:",
      round(float(embed(["heart", "attack"]) @ embed(["myocardial", "infarction"])), 3),
      "| TF-IDF:", round(float((tfv.transform(["heart attack"]) @ tfv.transform(["myocardial
infarction"]).T).toarray()[0, 0]), 3))
```

Error analysis — bằng chứng từ corpus

Context word (±5, bỏ stopwords) phổ biến nhất và một số bigram, cho các trường hợp được phân tích trong `error_analysis.md`.

```
for w in ["football", "soccer", "disease", "car", "automobile", "banana", "cayenne", "good", "bad"]:
    print(f"{w:10s} (count {freq[w]:5d}):", context_evidence(w, top=10))
print("
bigram good/bad + X:", {f"{a} {b}": bigr[(a, b)] for a in ("good", "bad") for b in ("idea", "luck", "news",
"taste")})
print("banana: 'knock' trong context =", dict(context_evidence("banana", top=30)).get("knock"), "| 'banana
republic' =", bigr[("banana", "republic")])
print(f"cos(banana, car) = {wv.similarity('banana', 'car'):.3f}, cos(banana, apple) = {wv.similarity('banana',
'apple'):.3f}, "
      f"cos(banana, fruit) = {wv.similarity('banana', 'fruit'):.3f}, cos(good, bad) = {wv.similarity('good',
'bad'):.3f}, cos(good, excellent) = {wv.similarity('good', 'excellent'):.3f}")
```

Polysemy — *bank*

Static Word2Vec có **một** vector cho *bank*, dù corpus dùng cả nghĩa tài chính và nghĩa bờ sông.

```
print("most_similar('bank'):", [(a, round(s, 2)) for a, s in wv.most_similar("bank", topn=10)])
for other in ["money", "financial", "loan", "river", "shore", "west"]:
    print(f"  cos(bank, {other:9s}) = {wv.similarity('bank', other):.3f}")
left = Counter(a for a, b in zip(words, words[1:]) if b == "bank")
print("Từ đứng ngay trước 'bank' trong text8:", left.most_common(12))
river_like = sum(left[w] for w in ["river", "west", "left", "south", "north", "east", "its", "opposite"])
fin_like = sum(left[w] for w in ["central", "world", "national", "investment", "reserve", "development",
"commercial", "savings"])
print(f"≈ nghĩa địa lý/bờ sông (river/west/left/... bank): {river_like} | ≈ nghĩa tài chính
(central/world/national/... bank): {fin_like}")
s1, s2 = "i deposited money in the bank".split(), "we sat on the river bank".split()
v1, v2 = wv["bank"], wv["bank"]
print("Vector của 'bank' trong hai câu giống hệt nhau:", np.array_equal(v1, v2))
ctx = lambda s: np.mean([wv[w] for w in s if w != "bank" and w in wv and w not in STOP], axis=0)
for s in (s1, s2):
    c = ctx(s)
    print(f"  context '{' '.join(s)}': cos(context, money) = {co.cosine_similarity(c, wv['money']):.2f},
cos(context, river) = {co.cosine_similarity(c, wv['river']):.2f}")
```

```
res = []
for name, r in arch.iterrows():
    for k, v in r.items(): res.append({"experiment": "architecture", "setting": name, "metric": k, "value": v})
for w, r in win_table.iterrows():
    for k, v in r.items(): res.append({"experiment": "window", "setting": w, "metric": k, "value": v})
for d, r in dim_table.iterrows():
    for k, v in r.items(): res.append({"experiment": "dimension", "setting": d, "metric": k, "value": v})
```

```
for w, r in exp1.iterrows():
    for k, v in r.items():
        if k != "matrix size": res.append({"experiment": "co-occurrence", "setting": w, "metric": k, "value": v})
for _, r in tiny_df.iterrows(): res.append({"experiment": "100-sentence corpus", "setting": f"seed {r.seed}",
"metric": "cos(doctor, physician)", "value": r["cos(doctor, physician)"]})
for _, r in sim_eval.iterrows(): res.append({"experiment": "word similarity", "setting": MAIN, "metric": r.pair,
"value": r.cosine})
for m, g in search_eval.groupby("method"):
    for k in ("P@5", "R@10", "RR"): res.append({"experiment": "semantic search", "setting": m, "metric": "MRR" if k
== "RR" else k, "value": g[k].mean()})
pd.DataFrame(res).to_csv("results.csv", index=False)
print("Đã ghi results.csv:", len(res), "dòng")
```

Error analysis — Word2Vec (Skip-gram, 100 chiều, window 5) trên text8

Số liệu lấy từ `word_embedding.ipynb`. "Đúng" = gần với trực giác con người; "sai/bất ngờ" = lệch trực giác. Bằng chứng corpus lấy từ đếm trực tiếp trong text8 (cửa sổ ±5, bỏ stopwords).

3 similarity đúng

Cặp	Cosine	Quan sát	Giải thích
computer → computers	0.80 (top-1)	computers là biến thể số nhiều	Cùng nghĩa, cùng context
car → automobile	0.777	automobile là top-2 của car	Đồng nghĩa; cả hai có context *race, driver*
disease → diseases / infection	0.85 / 0.82	top-3 neighbors của *disease* là *diseases, infectious, infection*	Biến thể hình thái và từ cùng trường nghĩa

3 similarity sai hoặc bất ngờ

Cặp	Cosine	Observed	Expected	Possible explanation	Evidence from corpus
doctor → supervillain / onizuka / hellboy	0.60	top-1, 3, 5 của *doctor* là nhân vật hư cấu	physician, nurse, patient	**Polysemy + domain bias**: *doctor* là tên nhân vật (*Doctor Who*) và học hàm (*juris doctor*)	"doctor who" 222 lần, "the doctor" 187 lần, "juris doctor" 16 lần; *medical doctor* chỉ 20 lần
banana → cayenne / daiquiri / lobster	0.73 / 0.71	banana gần đồ ăn/đồ uống	fruit, yellow, apple	**Context window + corpus nhỏ**: *banana* chỉ có 111 lần; context chủ yếu là công thức nấu ăn	"knock" xuất hiện 21 lần trong cửa sổ ±5 quanh *banana* (không phải cụm cố định); cụm "banana republic" 5 lần; không có "banana split"
good → bad	0.78 (cao)	good và bad gần nhau	trái nghĩa, cosine thấp	**Distributional limitation**: hai từ thay thế được trong cùng câu (*a good/bad idea*, *good/bad luck*)	"good luck" 39 lần, "bad luck" 45 lần; "good idea" 37, "bad idea" 14

Đáng chú ý: *doctor* và *physician* có cos 0.466, nhưng *physician* không nằm trong top-5 của *doctor* trên text8. Đây là một lỗi recall của corpus nhỏ, không phải lỗi thuật toán.

Ngoài ra *bank* cho *banks, monetary, fund* (nghĩa tài chính) với cos 0.65–0.75, và cos(bank, river) chỉ 0.44 dù *river bank* có trong corpus: đây là **polysemy** (một vector cho hai nghĩa; xem `reflection.md`).

Nguyên nhân tổng hợp

Nguyên nhân	Ví dụ
Polysemy	doctor (Doctor Who / bác sĩ), bank (ngân hàng / bờ sông)
Domain bias của corpus	text8 nhiều về phim, truyện, nhân vật hư cấu → *doctor* gần nhân vật
Frequency thấp	banana (111 lần), cayenne (18 lần): vector không ổn định
Context giống nhau nhưng nghĩa khác	good / bad: cùng context đánh giá

Nguyên nhân	Ví dụ
Window	window 5 lấy cả chủ đề rộng (food, recipe)

Đối với corpus arXiv (LAB 01 corpus), *doctor* cho *physician, clinician, physicians, nurse, patient* — đúng hơn nhiều, vì domain thay đổi. Vậy lỗi phần lớn đến từ **corpus**, không chỉ thuật toán.

Reflection — từ Word2Vec đến Transformer

Số liệu lấy từ `word_embedding.ipynb` / `results.csv` (model chính: Skip-gram, 100 chiều, window 5, text8).

Đối chiếu prediction

Prediction	Kết quả	Đúng/sai
P1: doctor–physician gần nhất; banana, car xa	doctor–physician 0.466 ≈ doctor–hospital 0.468; banana–car = **0.450**, cao hơn nhiều so với banana–doctor (0.038)	Đúng một phần: không có từ y tế nào "gần nhất" rõ rệt; banana–car gần nhau ngoài dự đoán
P2: window lớn → chủ đề, window nhỏ → chức năng	window 2 → 5 → 10: doctor–hospital 0.437 → 0.468 → 0.477 (tăng). Co-occurrence: window 1 cho *doctor* → *physician*, window 5 cho *doctor* → *patient, medical*	Đúng
P3: dimension không chắc chắn tăng chất lượng	50 / 100 / 300: WordSim 0.662 / **0.689** / 0.676; analogy 0.294 / 0.386 / 0.390; IMDb 0.723 / 0.754 / **0.785**; thời gian 55 / 52 / 207 s	Đúng: intrinsic bão hòa ở 100, downstream vẫn tăng
P4: 100 câu thì không chắc chắn	cos(doctor, physician) = 0.54 ± 0.04 qua 5 seed, nhưng top-3 neighbors của doctor là *fake, body, mystical, helper*	Đúng: con số có vẻ cao nhưng không gian không có nghĩa

Bảng so sánh representation

Representation	Context-dependent?	Sparse/Dense	Một từ có nhiều vector?
TF-IDF	Không (biểu diễn document, không biểu diễn nghĩa của từ)	Sparse (99.8% zeros ở LAB 01)	Không: một từ = một chiều cố định
Co-occurrence	Không (đếm trên toàn corpus)	Sparse (density 0.9–3.5% với V = 10 000)	Không: một hàng / từ
Word2Vec	Không (static, học từ context nhưng cố định sau khi train)	Dense (100 chiều)	Không: một vector / từ
Contextual embedding	**Có**: vector tính lại theo câu	Dense	**Có**: mỗi lần xuất hiện một vector

Tại sao *bank* cần contextual representation?

text8 dùng *bank* theo cả hai nghĩa: từ đứng trước là *west/left/south…* (khoảng 307 lần, nghĩa địa lý, kể cả West Bank) và *central/world/national…* (khoảng 499 lần, tài chính). Word2Vec gộp tất cả vào **một** vector, kéo về nghĩa phổ biến hơn: 10 neighbors gần nhất đều là tài chính (*banks, monetary, fund, banking, ecb…*), cos(bank, river) = 0.44 < cos(bank, financial) = 0.59. Vector của *bank* trong "I deposited money in the bank" và "we sat on the river bank" **giống hệt nhau**, dù chỉ riêng context của hai câu đã phân biệt được hai nghĩa (cos(context, money) = 0.76 so với 0.40; cos(context, river) = 0.41 so với 0.68). Một representation tính vector của từ *từ chính context đó* (contextual embedding, self-attention) sẽ tách được hai nghĩa.

Bài học rút ra

- **Distributional hypothesis hoạt động, nhưng "similar context" ≠ "synonym":** *good–bad* = 0.78 (trái nghĩa, cùng context); *doctor* gần *supervillain, hellboy* vì *doctor who* (222 lần) và truyện tranh.

- **Corpus quyết định nghĩa:** *transformer* trong text8 gần *resistor, mosfet*; trong arXiv gần *bert, xlnet, seq2seq*. *doctor* trong arXiv gần *physician, clinician*.
- **PPMI là bước quan trọng với count vectors:** raw count cho computer–river = 0.88 (vì cùng context *the, of*); PPMI giảm xuống 0.04.
- **Semantic search (LAB 01 → LAB 03):** avg embedding đưa relevant đầu tiên của *heart attack prediction* từ rank 18 lên rank 7 và tăng MRR (0.642 → 0.664), nhưng P@5 giảm (0.525 → 0.300). Hybrid lexical + dense cân bằng hơn (P@5 0.475, MRR 0.664). Lưu ý: nhãn relevance được tạo bằng pooling từ các hệ lexical nên có thể đánh giá thấp phương pháp dense. cos("heart attack", "myocardial infarction") = 0.60 với Word2Vec, so với 0 với TF-IDF.
- **Static embedding vẫn thiếu:** một vector cho mọi nghĩa, không xét thứ tự từ → động lực cho sequence models (LAB 05) và contextual embeddings / Transformer.